

Contents

重建平台技术报告—sqc/ 框架	2
0. 导言	2
0.1 平台定位	2
0.2 三条主线	2
0.3 共用基础设施	3
0.4 文档导航	4
主线 A •波形重建	4
1.1 共用基石: 相位 $\rightleftarrows$ 频率 $\rightleftarrows$ 磁通	4
1.2 Cryoscope —截断扫描重建波形	5
1.2.1 物理原理	5
1.2.2 实验层	5
1.2.3 重建层	5
1.2.4 配套标定 CryoscopeCalibration	6
1.2.5 Notebook 调用示例	7
1.2.6 约束与陷阱	7
1.3 Delay Ramsey —衰减拖尾测量	7
1.3.1 物理原理	7
1.3.2 实验层	8
1.3.3 重建层	8
1.3.4 配套标定 DelayRamseyCalibration	9
1.3.5 Notebook 调用示例	9
1.3.6 约束与陷阱	10
1.4 脉冲补偿—二维扫描的“自归零”测量	10
1.4.1 物理原理	10
1.4.2 实验层	10
1.4.3 重建层	11
1.4.4 Notebook 调用示例	11
1.4.5 约束与陷阱	12
1.5 瞬态磁场感知—滑动 Ramsey + 反卷积	12
1.5.1 物理原理	12
1.5.2 核函数估计 KernelEstimator	12
1.5.3 实验层	12
1.5.4 重建层	13
1.5.5 Notebook 调用示例	14
1.5.6 约束与陷阱	14
1.6 四种波形重建方法对比	15
主线 B •Qubit 频率标定	15
2.1 标定层概览	15
2.2 共用基石: Ramsey FFT 频率拟合	17
2.3 FluxResponseCalibration —扫 Φ 拟 $f(\Phi)$	17
2.4 FrequencyMeasurement —单点 f 测量	18
2.4.1 method="ramsey" ——Ramsey FFT 双模	18
2.4.2 method="transient" ——正交 Ramsey 差分 + 核函数 $G_{\perp}$	19
2.4.3 调用示例	19
2.4.4 通过 Scheduler 注册	20
2.4.5 约束与陷阱	20
2.5 SinglePointFrequencyCalibration —单点 f 调谐	20
2.5.1 顶层入口	21
2.5.2 Secant 法 (默认 step_method="secant")	21
2.5.3 Bisection 法 (step_method="bisection")	21
2.5.4 内嵌测频策略	22
2.5.5 输出 CalibrationTable 结构	22
2.5.6 Notebook 调用示例	22

2.5.7 通过 Scheduler 注册 (依赖注入)	23
2.5.8 未来扩展的预留口	23
2.5.9 约束与陷阱	23
主线 C · 预失真	24
3.1 失真模型层	24
3.1.1 SingleExponentialDistortion —单指数尾	24
3.1.2 MultiExponentialDistortion —K 个指数并联	24
3.1.3 FIR / IIR / Custom	24
3.2 控制线封装 ControllLine	25
3.3 标层 WaveformCalibration + PredistortionDesigner	25
3.3.1 WaveformCalibration ——统一入口	25
3.3.2 PredistortionDesigner ——三种逆滤波器设计	25
3.3.3 稳定性检查	26
3.4 端到端 workflow PredistortionValidationWorkflow	26
3.5 Z-线交叉串扰 ZCrosstalkWorkflow	27
3.6 Notebook 端到端示例 (§10)	27
3.6.1 §10.0 —设置 (cell 24)	27
3.6.2 §10.1 —Rabi Chevron 失真诊断 (cell 26)	28
3.6.3 §10.2 —两种阶跃响应测量方法	28
3.6.4 §10.3 —预失真滤波器设计 (cell 35)	28
3.6.5 §10.4 —三层验证 (cells 38, 40, 42)	28
3.7 约束与陷阱	28
附录	29
附录 A · 公式速查	29
附录 B · 参考文献	29
附录 C · 测试与回归索引	30
单元测试 (tests/unit/)	30
回归测试 (tests/regression/)	30
运行测试	30
附录 D · CONFIG 默认值 (关键超参)	30

重建平台技术报告—sqc/ 框架

内部技术文档·面向后续维护者与协作者对应代码版本: sqc/ v0.1.0 (refactor 已完成 Phase 1-5) Notebook 主导引:
Simulation_sqc.ipynb 配套架构文档: docs/architecture.md

0. 导言

0.1 平台定位

本平台 (sqc/) 是一套面向超导 Transmon qubit 的“端到端时变磁通感知 + 控制线标定 + AWG 预失真”仿真框架。所有时间使用 ns、所有频率使用 GHz·(2), 自然单位 $\hbar=1$ 。底层物理求解器为 QuTiP mesolve。

整套平台围绕一个**统一时间量子** $dt = 1 / \text{awg.sample_rate}$ 组织 (sqc/config.py:51-56), 所有时间轴一律用 np.arange(start, stop, dt) 派生 (项目硬约束 R9)。

0.2 三条主线

主线	目标	输入	输出	主要模块
A. 波形重建	反演未知磁通波形 $\Phi(t)$	qubit 测量数据 (p_e , $p_e I$, $p_e Q$)	FluxSignal (Φ vs t)	sqc/experiments/, sqc/reconstruction/
B. 频率标定	建立 $f(\Phi)$ 曲线 / 单点 f	qubit + flux 扫描	CalibrationTable ($\Phi \rightarrow f$ 或 $V \rightarrow f$)	sqc/calibration/

主线	目标	输入	输出	主要模块
C. 预失真	反求 AWG 输入使片上波形等于目标	测得的 $H()$ 或阶跃响应	逆 DistortionModel + 预补偿 AWG 波形	sqc/hardware/, sqc/calibration/waveform.py, sqc/workflows/

三条主线在数据上互相喂养:

主线 B (频率标定)
FluxResponse ($f(\Phi)$) / FrequencyMeasurement
(单点测量 ramsey/transient) /
SinglePointFrequencyCalibration (闭环调谐)

CalibrationTable(f_{phi} , f_{01})

主线 A (波形重建)
Cryoscope / DelayRamsey / PulseComp / Transient

 $\Phi(t)$ 重建结果 / kernel(t) / transfer fn

主线 C (预失真)
WaveformCalibration → PredistortionDesigner
→ PredistortionValidationWorkflow / ZCrosstalk

0.3 共用基础设施

所有协议、所有重建、所有标定共享 4 个抽象基类与一个全局配置单例:

类 / 模块	文件	作用
CONFIG (singleton)	sqc/config.py:233	全局配置: AWG、qubit 默认、pulse 时间轴、reconstruction 默认超参
Experiment (ABC)	sqc/experiments/base.py:10	每个协议的统一接口: build_sequence() + run() → ExperimentResult
Reconstruction (ABC)	sqc/reconstruction/base.py:1	重建算法接口: reconstruct(measurement, ...) → FluxSignal
Calibration (ABC)	sqc/calibration/base.py:167	标定接口: calibrate() → CalibrationTable
CalibrationTable (dataclass)	sqc/calibration/base.py:19	标定结果容器, 含 evaluate(x) 正向 + inverse(y) 反向插值
Workflow (ABC)	sqc/workflows/base.py:11	高层工作流: 编排多个 experiment + calibration + reconstruction
IQReadoutModel	sqc/hardware/readout.py:74	I/Q 双通道读出 (两次 Ramsey 序列, /2 phase offset)

CalibrationTable.evaluate / inverse 实现在 [sqc/calibration/base.py:63-164](#), 统一使用 scipy interp1d cubic/quadratic/linear 自动降级。

0.4 文档导航

- §1 = 主线 A (波形重建, 4 个协议)
- §2 = 主线 B (频率标定, 3 个标定器: FluxResponse / FrequencyMeasurement / SinglePointFrequencyCalibration, v2.5 起测量与调谐职责分离)
- §3 = 主线 C (预失真, 5 个失真模型 + 设计器 + 端到端 workflow)
- 附录 A = 公式速查
- 附录 B = 参考文献
- 附录 C = 测试与回归索引

每个协议的子章节统一按以下顺序写:

物理原理与目标 → 数学公式 → 实验层代码 (experiments/) → 重建层代码 (reconstruction/) → 配套标定 (如有) → Notebook 调用示例 → 关键超参与约束 → 局限与适用范围

主线 A • 波形重建

共四个子协议: Cryoscope、Delay Ramsey、**脉冲补偿**、**瞬态磁场**。它们都是把未知磁通 $\Phi(t)$ 反演出来, 但探测窗口与精度权衡不同。

1.1 共用基石: 相位 $\rightleftharpoons$ 频率 $\rightleftharpoons$ 磁通

Transmon 的基态-激发态频率随磁通的依赖关系 (Gao 2021 §III.B Eq. 13-14):

$$\omega_Q(\Phi) = \sqrt{8 E_J |\cos(\pi\Phi/\Phi_0)| E_C} - E_C$$

代码实现: [sqc/reconstruction/dispersion.py:11](#) `qubit_inverse_frequency()`, 给定 Δ 反求 Φ :

$$\cos(\pi\Phi) = \frac{(f_{\text{target}} + E_C)^2}{8 E_J E_C}, \quad \Phi = \frac{1}{\pi} \arccos(\text{clip}(\text{ratio}, 0, 1))$$

```
# sqc/reconstruction/dispersion.py:11-41
def qubit_inverse_frequency(dphi_dt: np.ndarray, qubit) -> np.ndarray:
    f_q = qubit.frequency
    f_target = f_q + dphi_dt
    EC = qubit.EC
    EJ0 = getattr(qubit, "EJ_0", qubit.EJ)
    ratio = (f_target + EC) ** 2 / (8.0 * EC * EJ0)
    ratio = np.clip(ratio, 0.0, 1.0)
    total_flux = np.arccos(ratio) / np.pi
    bias = getattr(qubit, "flux_bias", getattr(qubit, "flux", 0.0))
    return total_flux - bias
```

Ramsey-类协议 (Cryoscope、Delay Ramsey) 的相位累积公式 (Gao 2021 §V.B):

$$\varphi(t) = \int_0^t \Delta\omega(\tau) d\tau \approx \int_0^t [\omega_Q(\Phi_{\text{bias}} + h(\tau)) - \omega_d] d\tau$$

由此 $d/dt = \Delta(t)$, 再通过上面的 dispersion 反演就得到 $h(t)$ 。

I/Q 双通道相位提取在 [sqc/hardware/readout.py:97-160](#), 跑两次 Ramsey 序列 ($\text{phase2}=0 \rightarrow \text{I 通道}$; $\text{phase2}=\pi/2 \rightarrow \text{Q 通道}$):

$$p_e^I = \frac{1}{2}[1 + \cos \varphi], \quad p_e^Q = \frac{1}{2}[1 + \sin \varphi]$$

$$\varphi = \text{atan2}(0.5 - p_e^I, p_e^Q - 0.5)$$

```
# sqc/hardware/readout.py:122-160 (节选)
ctrl_I = create_ramsey_pulse(t_rabi, tau, omega_d=omega_d,
                             phase1=np.pi / 2, phase2=0.0, qubit=qubit)
ctrl_Q = create_ramsey_pulse(t_rabi, tau, omega_d=omega_d,
                             phase1=np.pi / 2, phase2=np.pi / 2, qubit=qubit)
# ... mesolve I 通道、Q 通道
return {"p_e_I": float(result_I.expect[0][-1]),
        "p_e_Q": float(result_Q.expect[0][-1])}
```

1.2 Cryoscope —截断扫描重建波形

Notebook 对应: §5 (基础 demo)、§10.2.1 (阶跃响应测量) **理论:** Rol 2020 Cryoscope (Appl. Phys. Lett. 116, 054001) · Gao 2021 §V.E

1.2.1 物理原理

施加待测的磁通波形 $\Phi(t)$, 在不同截断时刻 t_d 处把信号切掉, 使 qubit 在 $[0, t_d]$ 区间内自由演化, 累积相位:

$$\varphi(t_d) = \int_0^{t_d} \Delta\omega(\Phi(\tau)) d\tau$$

数值微分得到瞬时频率失谐:

$$\frac{d\varphi}{dt_d} = \Delta\omega(\Phi(t_d)) = 2\pi \Delta f(h(t_d))$$

最后用**两种反演策略之一**映射 $\Delta \rightarrow h$:

- "calibration" ——用 CalibrationTable.inverse() 查 h 标定表 (本质是 $(h) = \cdot h \cdot$ 的非线性版本)。
- "response" ——用解析 Transmon 色散 (§1.1) 反演 $\Delta \rightarrow \Phi$ 。

1.2.2 实验层

[sqc/experiments/cryoscope.py:26](#) CryoscopeExperiment:

```
@dataclass
class CryoscopeExperiment(Experiment):
    qubit: object
    flux_signal: FluxSignal | None = None
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
    tau: float = field(default_factory=lambda: CONFIG.reconstruction.cryoscope_tau) # 100 ns
    trunc_list: np.ndarray | None = None
    omega_d: float | None = None
```

run() 流程 ([sqc/experiments/cryoscope.py:76-137](#)):

1. 构造 IQReadoutModel(tau, t_rabi, omega_d)
2. 对每个截断时刻 $\text{trunc} \in \text{trunc_list}$:
 - $\text{phi_truncated} = \text{flux_signal.copy}(); \text{phi_truncated.truncate}(0, \text{trunc})$ ——把信号截到 $t=\text{trunc}$
 - $\text{qubit.qubit_in_mag}(\text{phi_truncated}, \text{frame}=1, \text{omega_d}=\dots)$ ——让 qubit 看到截断后的磁通
 - $\text{readout.measure}(\text{qubit}) \rightarrow \text{p_e_I}, \text{p_e_Q}$
3. 相位提取:

```
# sqc/experiments/cryoscope.py:117-118
varphi = np.arctan2(0.5 * p_e_I, p_e_Q - 0.5)[::-1]
varphi = np.unwrap(varphi)
```

输出 ExperimentResult(data={"varphi", "p_e_I", "p_e_Q"}, axes={"trunc"}).

1.2.3 重建层

[sqc/reconstruction/cryoscope.py:28](#) CryoscopeReconstruction:

```
@dataclass
class CryoscopeReconstruction(Reconstruction):
    tau: float = field(default_factory=lambda: CONFIG.reconstruction.cryoscope_tau)
```

```

inversion: Literal["calibration", "response"] = "calibration"
calibration: CalibrationTable | None = None
qubit: object | None = None
use_sg_filter: bool = False
sg_window: int = 7
sg_poly: int = 2

```

核心算法 ([sqc/reconstruction/cryoscope.py:65-105](#)):

```

def reconstruct(self, measurement, kernel=None, calibration=None, dt=None) -> FluxSignal:
    varphi = np.asarray(measurement.data["varphi"], dtype=float)
    trunc = np.asarray(measurement.axes["trunc"], dtype=float)
    if len(trunc) > 1 and trunc[0] > trunc[-1]:
        trunc = trunc[::-1]
    if dt is None:
        dt = float(trunc[1] - trunc[0])

    if self.use_sg_filter:
        from scipy.signal import savgol_filter
        dphi_dt = savgol_filter(varphi, window_length=self.sg_window,
                                polyorder=self.sg_poly, deriv=1, delta=dt)
    else:
        dphi_dt = np.gradient(varphi, trunc)

    if self.inversion == "calibration":
        phi_equiv = dphi_dt * self.tau
        h_recon = cal.inverse(phi_equiv)
    elif self.inversion == "response":
        h_recon = qubit.inverse_frequency(dphi_dt, self.qubit)

    return FluxSignal(type=8, t_list=trunc, signal=h_recon)

```

关键点: $\phi_{\text{equiv}} = d\phi_{\text{dt}} * \tau$ 把瞬时频率失谐 Δ 换算成一个等价的 “-长 Ramsey 相位”，再走 [CalibrationTable.inverse\(\)](#) 查表。这里 必须与标定时使用的 严格一致 (标定时同样跑 -长 square pulse + IQ Ramsey)。

1.2.4 配套标定 CryoscopeCalibration

[sqc/reconstruction/cryoscope.py:113](#) CryoscopeCalibration:

```

@dataclass
class CryoscopeCalibration(Calibration):
    qubit: object
    h_list: np.ndarray | None = None
    tau: float = 100.0
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())

```

calibrate() 流程 ([sqc/reconstruction/cryoscope.py:141-180](#)):

1. 对每个 $h \in h_list$ 构造 -长 square pulse:

```

t_sig = CONFIG.pulse.make_time(0, t_total)
signal[mask] = float(h)
Phi = FluxSignal(type=8, t_list=t_sig, signal=signal)

```

$t_total = 2 * t_pi2 + \tau$
mask 是 $[t_pi2, t_pi2 + \tau]$

2. IQ Ramsey 读取 $\rightarrow p_{e-I}, p_{e-Q}$

3. 计算原始相位 $\text{varphi_raw} = \arctan2(0.5 - p_{e-I}, p_{e-Q} - 0.5)$

4. **关键:** 基于 Transmon 解析模型反推 2 跳变次数 ([sqc/reconstruction/cryoscope.py:182-194](#)):

```

omega_q = np.sqrt(8.0 * EJ0 * np.abs(np.cos(np.pi * total_flux))) * EC - EC
varphi_theory = (omega_q - omega_d) * self.tau
n_wraps = np.round((varphi_theory - varphi_raw) / (2.0 * np.pi))
return varphi_raw + 2.0 * np.pi * n_wraps

```

这是把 $\arctan2$ 输出的 $\pm$ 包裹放回正确的分支, 用物理模型驱动 unwrap 而非纯数值 unwrap——后者在大相位 ($>$) 时容易跳错。

5. 返回 `CalibrationTable(kind="phi_h", inputs=h_list, outputs=unwrapped_phi)`。

1.2.5 Notebook 调用示例

完整 pipeline ([Simulation_sqc.ipynb](#) §5, cell 12):

```
# === Cryoscope: 标定 → 测量 → 重建 ===
qubit = TransmonQubit(
    EC=0.2 * 2 * np.pi, EJ=10.0 * 2 * np.pi,
    T1=100.0e3, T2=50.0e3,
    flux=np.arctan(np.sqrt(2)) / np.pi, # 这是 legacy "甜点" 工作点
    n_levels=3,
)

# Step 1: 标定
cal = CryoscopeCalibration(qubit=qubit,
                           h_list=np.linspace(-0.03, 0.03, 21),
                           tau=50.0)
cal_table = cal.calibrate()

# Step 2: 测量
test_signal = FluxSignal(type=3, t_list=CONFIG.pulse.make_time(0, 80),
                          amplitude=0.01, center=40, width=5)
cryo_exp = CryoscopeExperiment(qubit=qubit, flux_signal=test_signal, tau=50.0)
meas = cryo_exp.run()

# Step 3: 重建 (两种 inversion 对比)
recon_cal = CryoscopeReconstruction(calibration=cal_table, tau=50.0, inversion="calibration")
recon_resp = CryoscopeReconstruction(qubit=qubit, tau=50.0, inversion="response")
h_cal = recon_cal.reconstruct(meas)
h_resp = recon_resp.reconstruct(meas)
```

§10.2.1 阶跃响应特殊用法 (cell 29): 把 flux 工作点从甜点改到敏感点 (flux=0.25), 因为甜点处 $d/d\Phi = 0$ (二次响应), (h) 是对称抛物线, 1D inverse 区分不出 $\pm h$ 。敏感点处 (h) 单调, 可以反演。

```
qubit_cryo = TransmonQubit(... flux=0.25, n_levels=3) # ← 敏感点
step_amp = 0.001 # 选小幅度保证 |_max| < , 避免 unwrap 风险
```

1.2.6 约束与陷阱

约束	出处	说明
必须与标定一致	cryoscope.py:55-56 vs :132	否则 $\phi_{\text{equiv}} = d\phi_{\text{dt}} * \tau$ 量纲对不上
flux 工作点必须敏感 ($d/d\Phi \neq 0$)	notebook cell 29	甜点 (h) 是抛物线, 无法 1D 反演
$ _max < \dots$	notebook §10.2.1	否则需要更激进的 unwrap (基于物理模型那种)
arctan2 顺序: $(0.5 - p_e^I, p_e^{Q-0.5})$	cryoscope.py:117	与 src 历史保持一致; 调换顺序会得到 —
trunc_list 默认倒序 [140:20:-1]	cryoscope.py:70	与 src/protocal.py case 5 legacy 一致; 输出 varphi 再 [::-1] 翻回

1.3 Delay Ramsey — 衰减拖尾测量

Notebook 对应: §7 **物理:** 用短 Ramsey 序列在阶跃下降沿后滑动采样, 重建 flux pulse 的拖尾。

1.3.1 物理原理

施加方波 flux 脉冲, 其下降沿后存在指数拖尾 (IIR-type 失真)。在下降沿后延迟 t_d 处插入一段长度为 τ_R 的 Ramsey 序列:

$$\varphi(t_d) = \int_{t_d}^{t_d + \tau_R} \Delta\omega(\tau) d\tau \approx \tau_R \cdot \Delta\omega(\bar{\Phi}_{\text{tail}}(t_d))$$

近似式假设 τ_R 足够短, 拖尾在窗内近似常数。此式直接给出 “ **τ_R -窗口平均**” 而非瞬时值 (窗口越长, 时间分辨率越差)。

线性化 (小 h 极限):

$$\varphi(t_d) \approx \tau_R \cdot \kappa \cdot \bar{h}(t_d)$$

其中 $\kappa = d/d\Phi$ 是 frequency sensitivity (`src/qubit.py` frequency_sensitivity).

1.3.2 实验层

`src/experiments/delay_ramsey.py:40` DelayRamseyExperiment:

```
@dataclass
class DelayRamseyExperiment(Experiment):
    qubit: object
    flux_signal: FluxSignal | None = None
    t_d_list: np.ndarray | None = None
    tau_R: float = field(default_factory=lambda: CONFIG.reconstruction.delay_ramsey_tau) # 20 ns
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
    omega_d: float | None = None
    run_baseline: bool = False # 默认 off: 零信号时 arctan2(0,0) 不稳
    t_fall: float = 0.0 # 下降沿位置, t_d 相对此参考
```

`run()` 流程 (`src/experiments/delay_ramsey.py:93-203`) 关键步骤:

1. 时间网格 `t_sig = CONFIG.pulse.make_time(0, 2*t_pi2 + tau_R)`
2. 可选 baseline —— 零 flux 跑一次记录 `p_e_I_base, p_e_Q_base` (用 I/Q 复数空间减, 避开 `arctan2(0,0)`)
3. 对每个 `t_d ∈ t_d_list`:
 - 把 flux 窗口化到自由演化区间 `[t_pi2, t_pi2+tau_R]` ($\pi/2$ 脉冲期间必须置零, 否则 flux 会失谐脉冲本身):

```
# src/experiments/delay_ramsey.py:134-143
signal = np.zeros(len(t_sig), dtype=float)
free_mask = (t_sig >= t_pi2_end) & (t_sig <= t_pi2_end + tau_R)
signal[free_mask] = np.array(
    [self.flux_signal.value_at(self.t_fall + t_d + float(t))
     for t in t_sig[free_mask]],
    dtype=float,
)
```

- `qubit.qubit_in_mag(phi_windowed, frame=1, omega_d=omega_d)`
 - `readout.measure(qubit) → p_e_I, p_e_Q`
4. 相位提取 + 复数空间 baseline 相减 (`delay_ramsey.py:157-178`):

```
varphi_base = float(np.arctan2(0.5 - p_e_I_base, p_e_Q_base - 0.5))
varphi_shifted = np.arctan2(0.5 - p_e_I, p_e_Q - 0.5) - varphi_base
varphi_shifted = (varphi_shifted + np.pi) % (2 * np.pi) - np.pi # wrap 回 [-, ]
varphi = np.unwrap(varphi_shifted)
```

这一步比直接 `unwrap` 原始相位更稳——大 `t_d` 处信号衰减, 原始 `arctan2(0,0)` 在 $\sim 3/4$ 噪声底飘移, 会污染 `unwrap` 的 2 分支判断。

1.3.3 重建层

`src/reconstruction/delay_ramsey.py:26` DelayRamseyReconstruction:

```
@dataclass
class DelayRamseyReconstruction(Reconstruction):
    inversion: Literal["response", "calibration"] = "response"
    qubit: object | None = None
    calibration: CalibrationTable | None = None
    tau_R: float | None = field(default_factory=lambda: CONFIG.reconstruction.delay_ramsey_tau)
```

两种反演路径:

A. `inversion="response"` —— 用 Transmon 解析色散反演 (`delay_ramsey.py:65-72`):


```
def _via_response(self, varphi, t_axis, tau) -> FluxSignal:
    phi = np.asarray(varphi, dtype=float).copy()
    if np.median(phi) > 0:
        phi = phi - 2.0 * np.pi # 防止整体平移到错的分支
    h = qubit.inverse_frequency(phi / tau, self.qubit)
    return FluxSignal(type=8, t_list=t_axis, signal=h)
```

B. inversion="calibration" ——用预先标定的 (z) 表反查 (delay_ramsey.py:74-83):

```
def _via_calibration(self, varphi, t_axis, cal) -> FluxSignal:
    phi = np.asarray(varphi, dtype=float).copy()
    cal_center = 0.5 * (cal.outputs.min() + cal.outputs.max())
    shift = cal_center - np.median(phi)
    n2pi = np.round(shift / (2.0 * np.pi))
    phi = phi + n2pi * (2.0 * np.pi) # 把测量相位平移到标定表的覆盖区
    flux = cal.inverse(phi)
    return FluxSignal(type=8, t_list=t_axis, signal=flux)
```

1.3.4 配套标定 DelayRamseyCalibration

sqc/reconstruction/delay_ramsey.py:91 DelayRamseyCalibration ——与重建器拍配套, 扫描已知 flux 高度 z, 拟合斜率 k:

$$\varphi_{\text{cal}}(z) = k \cdot z + \text{intercept}, \quad k = \tau_R \cdot \kappa$$

calibrate() (delay_ramsey.py:126-175) 关键步骤:

1. 先在零 flux 跑一次 baseline, 得 varphi_base
2. 对每个 $z \in z_list$ (默认 linspace(-0.02, 0.02, 41)) 施加方波 flux, 跑 IQ Ramsey 测
3. 同样做复数空间 baseline 相减 + wrap + unwrap
4. 一阶多项式拟合 k, intercept = np.polyfit(z_list, varphi, 1)
5. 返回 CalibrationTable(kind="phi_z", inputs=z_list, outputs=varphi_unwrapped, fit_params={"k": k, ...})

1.3.5 Notebook 调用示例

完整 pipeline (Simulation_sqc.ipynb §7, cell 16):

```
qubit = TransmonQubit(EC=0.2*2*np.pi, EJ=10.0*2*np.pi, T1=100e3, T2=50e3,
                      flux=0.25, n_levels=2)
```

```
tau_R = 20.0
```

```
# Step 1: 构造一个带指数拖尾的方波 (待测信号)
t_list = CONFIG.pulse.make_time(0, 100)
sig = np.zeros(len(t_list))
sig[(t_list >= 10) & (t_list <= 40)] = 0.008
sig[t_list > 40] = 0.008 * np.exp(-(t_list[t_list > 40] - 40) / 20)
test_flux = FluxSignal(type=8, t_list=t_list, signal=sig)
```

```
# Step 2: 标定 (z) 斜率
cal = DelayRamseyCalibration(qubit=qubit, z_list=np.linspace(-0.010, 0.010, 41),
                             tau_R=tau_R, t_rabi=np.linspace(0, 10, 20))
cal_table = cal.calibrate()
k = cal_table.fit_params["k"] # rad/φ
```

```
# Step 3: 测量 (启用 baseline 相减)
exp = DelayRamseyExperiment(qubit=qubit, flux_signal=test_flux,
                             t_d_list=CONFIG.pulse.make_time(0, 60)[:2],
                             tau_R=tau_R, t_fall=40.0, run_baseline=True)
meas = exp.run()
```

```
# Step 4: 重建 (用标定查表反演)
recon = DelayRamseyReconstruction(inversion="calibration", calibration=cal_table)
tail_flux = recon.reconstruct(meas)
```

1.3.6 约束与陷阱

约束	出处	说明
必须从下降沿后开始扫 t_d	$t_{fall}=40.0$ notebook	否则 flux 还在 hold 阶段, 测的不是拖尾
τ_R 必须远短于拖尾时间常数	物理	否则窗口平均把拖尾抹平; 典型 $\tau_R = 10-30$ ns
自由演化窗内才有 flux, $\tau/2$ 期间必须置零	delay_ramsey.py:134-143	否则 flux 失谐 $\tau/2$ 脉冲
baseline 相减必须在复数空间做	delay_ramsey.py:171-176	否则大 t_d 处的 $\arctan2(0,0)$ 噪声会污染 unwrap
重建出的是窗口平均 瞬时 Φ	推导式	notebook §7 提供了 τ_R -window moving average 作为真值参考

1.4 脉冲补偿—二维扫描的“自归零”测量

Notebook 对应: §8 **物理:** 拖尾使 qubit 失谐; 施加一个反向补偿 flux z 使其归零, 脉冲翻转概率 P_e 达极大。

1.4.1 物理原理

设拖尾 flux 在 $[-T, +T]$ 区间内平均为 $\langle \Phi_{tail} \rangle$, 叠加一个常数补偿 z 后总失谐为 $(\langle \Phi_{tail} \rangle + z)$ 。脉冲只有在失谐 0 时才能完美翻转 $|0\rangle \rightarrow |1\rangle$ 。扫描 z 找极大:

$$z^*(\tau) = \arg \max_z P_e(\tau, z) \implies \Phi_{tail}(\tau) \approx -z^*(\tau)$$

注意: 由于 脉冲宽度 $T > 0$, z^* 实际上是 $[-T, +T]$ 窗口的反向平均, **不是瞬时拖尾**。指数拖尾的衰减时间常数仍能被保留, 但幅度会被窗口平均稀释。

1.4.2 实验层

[sqc/experiments/pi_pulse_comp.py:42](#) PiPulseCompensationExperiment:

```
@dataclass
class PiPulseCompensationExperiment(Experiment):
    qubit: object
    flux_signal: FluxSignal | None = None
    tau_list: np.ndarray | None = None          # delay 扫描
    z_list: np.ndarray | None = None           # 补偿幅度扫描
    T_pi: float = field(default_factory=lambda: CONFIG.reconstruction.pi_pulse_T_pi) # 10 ns
    omega_bias: float | None = None
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
    t_fall: float = 0.0
```

run() 流程 ([pi_pulse_comp.py:97-203](#)) 核心:

```
for i, tau in enumerate(self.tau_list):
    for j, z in enumerate(self.z_list):
        # 拖尾 + 补偿 z (整个 脉冲期间)
        tail = np.array(
            [self.flux_signal.value_at(self.t_fall + tau + float(t))
             for t in t_sig], dtype=float)
        signal = tail + float(z)

        phi_composite = FluxSignal(type=8, t_list=t_sig, signal=signal)
        self.qubit.qubit_in_mag(phi_composite, frame=1, omega_d=self.omega_bias)

        # 同时加 脉冲 (在 t_rabi 时间窗内)
        H_pi = create_pulse(self.qubit, frame=1, type=1, t_list=t_sig,
                             omega_d=self.omega_bias, phase=0.0, angle=np.pi)
        H_total = (
            QobjEvo(self.qubit.H_list, tlist=self.qubit.mag_signal.t_list, order=1)
            + H_pi
```

```
)
result = mesolve(H_total, self.qubit.state, t_sig, [],
                 e_ops=[psi_e * psi_e.dag()])
p_e_2d[i, j] = float(result.expect[0][-1])
```

提取 z^* 时用抛物线顶点做子分辨率插值 (pi_pulse_comp.py:167-182):

```
for i in range(n_tau):
    j = int(np.argmax(p_e_2d[i]))
    if 0 < j < n_z - 1:
        pl, pc, pr = p_e_2d[i, j-1], p_e_2d[i, j], p_e_2d[i, j+1]
        denom = pr - 2.0 * pc + pl
        if abs(denom) > 1e-15:
            z_star[i] = self.z_list[j] - 0.5 * dz * (pr - pl) / denom
        else:
            z_star[i] = self.z_list[j]
    else:
        z_star[i] = self.z_list[j]
```

这避免了 z_list 分辨率太粗造成的“台阶状”重建结果。

1.4.3 重建层

sqc/reconstruction/pi_pulse_comp.py:21 PiPulseCompReconstruction:

```
@dataclass
class PiPulseCompReconstruction(Reconstruction):
    def reconstruct(self, measurement, **kwargs) -> FluxSignal:
        z_star = np.asarray(measurement.data["z_star"], dtype=float)
        t_axis = np.asarray(measurement.axes["tau"], dtype=float)
        return FluxSignal(type=8, t_list=t_axis, signal=-z_star)
```

——实际上没有反演：拖尾 = $-z^*$ ，已经在物理意义上是 1:1 的。

1.4.4 Notebook 调用示例

完整 pipeline (Simulation_sqc.ipynb §8, cell 18):

```
qubit = TransmonQubit(EC=0.2*2*np.pi, EJ=10.0*2*np.pi, T1=100e3, T2=50e3,
                      flux=0.25, n_levels=2)
omega_bias = qubit.frequency
```

```
# 与 §7 共用同一个待测信号 (一段方波 + 拖尾)
t_list = CONFIG.pulse.make_time(0, 100)
test_signal_arr = np.zeros(len(t_list))
test_signal_arr[(t_list >= 10) & (t_list <= 40)] = 0.008
test_signal_arr[t_list > 40] = (
    0.008 * np.exp(-(t_list[t_list > 40] - 40) / 20)
    + 0.008 * np.sin(2*np.pi * (t_list[t_list > 40] - 40) / 100)
)
test_flux = FluxSignal(type=8, t_list=t_list, signal=test_signal_arr)
```

```
# 2D 扫描
ppc_exp = PiPulseCompensationExperiment(
    qubit=qubit, flux_signal=test_flux,
    tau_list=CONFIG.pulse.make_time(0, 60)[:4],
    z_list=np.linspace(-0.012, 0.008, 17),
    T_pi=10.0, omega_bias=omega_bias,
    t_rabi=np.linspace(0, 10, 20),
    t_fall=40.0,
)
result = ppc_exp.run()
```

```
# 重建:  $\phi\_tail = -z^*$ 
recon = PiPulseCompReconstruction()
tail_flux = recon.reconstruct(result)
```

1.4.5 约束与陷阱

约束	出处	说明
z 扫描范围必须覆盖真实 $-\langle \Phi_{\text{tail}} \rangle$	物理	否则 argmax 在边界, 子分辨率插值无效
z 分辨率 dz 决定原始分辨率	pi_pulse_comp.py:170	抛物线插值能改善 $\sim 5\times$, 但仍受 dz 限制
测的是窗口平均 瞬时	1.4.1 推导	与 Delay Ramsey 同样的窗口效应, 但窗长是 $T_{\text{--}}$ 而非 T_{R}
复杂度 $O(n_{\text{tau}} \cdot n_z)$	算法	单次 mesolve $\times 21 \times 17 \sim 357$ 次; 最贵
不需要相位 unwrap	无	直接看 $P_{\text{--e}}$ 峰, 相位整圈跳变不影响 argmax

1.5 瞬态磁场感知—滑动 Ramsey + 反卷积

Notebook 对应: §4, §10.2.2 **物理:** 把 Ramsey 序列当一个“探针”滑过未知信号, 测得到的是信号与探针 kernel 的卷积。反卷积恢复信号。

1.5.1 物理原理

控制脉冲的“等效核函数” $k(t)$ 描述了它对每个时刻磁通的灵敏度。在小信号近似下:

$$\Delta p_e(t_d) = (k * B)(t_d) = \int k(t_d - \tau) B(\tau) d\tau$$

——即测量信号是真实信号 $B(t)$ 与核 $k(t)$ 的卷积。

反演策略三选一:

方法	公式	何时用
Wiener	$\hat{B}(\omega) = \frac{H^*(\omega) \Delta P(\omega)}{ H(\omega) ^2 + \lambda^2}$	线性区间, 最快
Hammerstein-Wiener	Wiener 出 $\Delta \rightarrow$ 解析 Transmon 反推 Φ	大信号、非线性
Levenberg-Marquardt	基函数参数化 B + 全密度矩阵正向仿真 + 牛顿-高斯	数据贵、精度要求高

1.5.2 核函数估计 KernelEstimator

[sqc/reconstruction/kernel.py:27](#) 统一替换了 src 的三处重复实现 (Pulse.get_kernel、CompositePulse.get_kernel、Analysis.get_kernel):

```
@dataclass
class KernelEstimator:
    stim_amplitude: float = field(default_factory=lambda: CONFIG.reconstruction.stim_amplitude) # 0.0215
    stim_width: float = field(default_factory=lambda: CONFIG.reconstruction.stim_width) # 3.0 ns
    auto_calibrate: bool = False
```

estimate(pulse, qubit) 流程 ([kernel.py:53-149](#)):

1. 跑一次基线 (无刺激) $\rightarrow p_{\text{e_base}}$
2. 对每个时刻 t_i 注入窄 Gaussian flux 刺激 (amplitude=0.0215, width=3 ns, center= t_i), 跑一次 $\rightarrow p_{\text{e_stim}}$
3. $\text{kernel}[i] = (p_{\text{e_stim}} - p_{\text{e_base}}) / \text{stim_area}$, 其中 $\text{stim_area} = \int \text{stim}(t) dt$

这是 Mehmet Canturk 式的“用窄 Gaussian 当 δ -函数”做线性响应函数测量。auto_calibrate=True 时按 $\kappa = d/d\Phi$ 自动调 amplitude 让频率漂移 $\sim 1\% \times |\text{anharmonicity}|$ ([kernel.py:151-176](#))。

1.5.3 实验层

[sqc/experiments/transient.py:24](#) TransientSensingExperiment:

```
@dataclass
class TransientSensingExperiment(Experiment):
    qubit: object
    flux_signal: FluxSignal | None = None
    flux_signal_zero: FluxSignal | None = None # 0-flux 参考
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
    omega_d: float | None = None
    scan_list: np.ndarray | None = None
```

run() 流程 (transient.py:89-144):

1. 构造 Ramsey 控制脉冲 create_ramsey_pulse(t_rabi, tau=0.0, omega_d, qubit) (tau=0 关键——让两段 $\pi/2$ 紧贴)
2. 用 SlidingMeasurementRunner 把 Ramsey 在 flux_signal 上滑动 $\rightarrow$ p_e(scan)
3. 同样在零 flux 上滑动 $\rightarrow$ p_e_base(scan)
4. $\Delta_p = p_e - p_{e_base}$
5. 同时调用 control_pulse.get_kernel(qubit) 计算 kernel 与 t_samples

输出 ExperimentResult(data={"p_e", "delta_p", "kernel", "flux_samples"}, axes={"scan", "t_samples", "t_flux"}).

1.5.4 重建层

sqc/reconstruction/transient.py:45 TransientReconstruction 支持三种 method:

A. Wiener 反卷积 transient.py:124-157:

```
def _reconstruct_wiener(self, measurement, kernel, dt=None, **_) -> FluxSignal:
    delta_p = np.asarray(measurement.data["delta_p"], dtype=float)
    if dt is None:
        scan = np.asarray(measurement.axes["scan"])
        dt = scan[1] - scan[0]

    N_del = len(delta_p); N_ker = len(kernel)
    N = N_del - N_ker + 1
    N_fft = N_del

    p_pad = np.zeros(N_fft); p_pad[:N_del] = delta_p
    k_pad = np.zeros(N_fft); k_pad[:N_ker] = kernel

    Y = np.fft.fft(p_pad)
    H = np.fft.fft(k_pad)
    G = np.conj(H) / (np.abs(H) ** 2 + self.lambda_reg ** 2) # Wiener 滤波器
    X_w = Y * G / dt
    x_rec = np.real(np.fft.ifft(X_w))[:N]

    return FluxSignal(type=8,
                      t_list=np.linspace(0, (N - 1) * dt, N),
                      signal=x_rec)
```

正则化 由 CONFIG.reconstruction.lambda_reg = 10.0 默认控制; 小信号、阶跃响应时常需要调到 ~ 50 抑制 ringing (notebook §10.2.2 cell 31)。

B. Hammerstein-Wiener transient.py:163-182: 先用 Wiener 得到等效频率失谐 (t) , 再用解析 Transmon 色散反推磁通:

```
def _reconstruct_hammerstein(self, measurement, kernel, dt=None, **_) -> FluxSignal:
    omega_signal = self._reconstruct_wiener(measurement, kernel, dt=dt)
    omega = np.asarray(omega_signal.signal, dtype=float)

    EC, EJ, freq = _get_qubit_params(self.qubit)
    ratio = np.clip((omega + freq + EC) ** 2 / (8 * EC * EJ), 0.0, 1.0)
    B = (1.0 / np.pi) * np.arccos(ratio)

    bias = getattr(self.qubit, "flux_bias", getattr(self.qubit, "flux", 0.0))
    B = B - bias # 减掉工作点 bias, 得  $h = \Phi - \Phi_{bias}$ 
```

```
return FluxSignal(type=8, t_list=omega_signal.t_list.copy(), signal=B)
```

C. Levenberg-Marquardt [transient.py:188-490](#) ——把 $B(t)$ 用基函数展开 $B(t) = \sum_k b_k \phi_k(t)$, 最小化残差 $\|p_{\text{meas}} - p_{\text{sim}}(b)\|^2$ 求 b :

$$b^{(n+1)} = b^{(n)} + \left(J^T J + \mu I + \lambda R \right)^{-1} J^T (p_{\text{meas}} - p_{\text{sim}}(b^{(n)}))$$

其中 R 是 smoothness 正则化矩阵 (regularization_matrix, [sqc/reconstruction/basis.py:138-170](#))。

基函数生成在 [basis.py:20-90](#), 三种选项 bspline / fourier / legendre. fourier 对应物理上常见的“频谱稀疏”先验, bspline 对应“光滑”先验, legendre 对应“低多项式”先验。

Jacobian 有两种实现:

- 伴随法 (adjoint, 默认 use_adjoint=True) ——[transient.py:301-390](#). 物理上是把测量算符 $\rho_0 = |1\rangle\langle 1|$ 反向演化到 $t_{\text{evolve}}[0]$, 再与正向演化的密度矩阵 $\rho(t)$ 做迹运算:

$$J_{ik} = \int_0^T \text{Tr} \left[\lambda(t) [G, \rho(t)] \right] \frac{\partial \Delta\omega}{\partial \Phi} \phi_k(t) dt$$

其中 $G = \text{qubit.n}$ (数算符)。

- 有限差分 ——[transient.py:396-420](#). 慢但稳。

正向仿真在 [_forward_simulation](#): 对每个 t_{delay} 单独跑一次 `mesolve(H_total, qubit.state, t_evolve, ...)`。

已知问题: adjoint Jacobian 与有限差分的偏差比预期大 (master TODO 0.3), 但 LM 仍能收敛——只是迭代次数偏多。生产环境推荐先用 Wiener / Hammerstein 做 warm start, 再用 LM 精修。

1.5.5 Notebook 调用示例

基础 demo (§4, cell 10):

```
qubit = TransmonQubit(EC=0.2*2*np.pi, EJ=10.0*2*np.pi, T1=100e3, T2=50e3,
                      flux=0.9553, n_levels=2) # 几乎到 sweet spot 附近
trans_exp = TransientSensingExperiment(qubit=qubit)
trans_result = trans_exp.run()
kernel = trans_result.data["kernel"]
```

```
rec = TransientReconstruction(method="wiener",
                              lambda_reg=CONFIG.reconstruction.lambda_reg)
B_rec_signal = rec.reconstruct(trans_result, kernel=kernel)
```

阶跃响应测量 (§10.2.2, cell 31):

```
qubit_sens = qubit_cryo # 复用 §10.2.1 的敏感点 qubit (flux=0.25)
t_scan = CONFIG.pulse.make_time(0, 70)
trans_exp = TransientSensingExperiment(
    qubit=qubit_sens,
    flux_signal=on_chip_flux, # 来自 §10.2.1 的失真后片上磁通
    scan_list=t_scan,
)
trans_result = trans_exp.run()
kernel = trans_result.data["kernel"]

lambda_reg = 50.0 # 阶跃信号弱 → 强正则化抑 ringing
rec_wiener = TransientReconstruction(method="wiener", lambda_reg=lambda_reg)
rec_hammer = TransientReconstruction(method="hammerstein",
                                     lambda_reg=lambda_reg, qubit=qubit_sens)

B_wiener = rec_wiener.reconstruct(trans_result, kernel=kernel)
B_hammer = rec_hammer.reconstruct(trans_result, kernel=kernel)
```

1.5.6 约束与陷阱

约束	出处	说明
Wiener 假设线性	推导 1.5.1	大信号下相位被压 ($\cos 1 - 2/2$), 需要 Hammerstein
<code>_reg</code> 决定噪声/分辨率权衡	<code>CONFIG.reconstruction.lambda_reg</code>	噪声大或信号弱时 $\uparrow$ (如 §10.2.2 用 50.0)
kernel 必须重新算	<code>TransientSensingExperiment.run()</code> 末尾	qubit 参数变了 kernel 也变
<code>stim_amplitude</code> 0.0215 是 legacy 默认	kernel.py:46	qubit 灵敏度变化时考虑 <code>auto_calibrate=True</code> 或显式覆盖
LM 启动需要 <code>qubit_in_mag</code> 已调用	transient.py:411,419,437	<code>adjoint</code> 计算依赖 <code>qubit.freq_coeffs</code>
LM Jacobian 偏差已知	TODO 0.3	不影响收敛, 影响一阶 step 大小

1.6 四种波形重建方法对比

来源: notebook §10.2.3 (cell 33) 总结 + 本文档对各方法的代码层分析。

维度	Cryoscope	Delay Ramsey	-pulse Comp.	Transient (Wiener)
测量量	(<code>t_trunc</code>)	(<code>t_d</code>)	$P_e(\cdot, z)$	$\Delta p_e(\text{scan})$
解相位	atan2 + 物理-driven unwrap	atan2 + baseline 减 + unwrap	argmax + 抛物线插值	不需要
时间分辨率	一个截断采样	<code>_R</code> 移动窗	<code>T_</code> 移动窗	AWG dt
幅度分辨率	标定查表 / 解析反演	<code>k_z</code> + 标定查表 / 解析反演	<code>dz · 0.5 ×</code> 插值	<code>_reg</code> / SNR
测一次的代价	$1 \times \text{mesolve} / \text{trunc}$	$2 \times \text{mesolve} / t_d$ (含 baseline)	$n_z \times \text{mesolve} /$	$2 \times n_{\text{scan}} \text{mesolve}$ (含 baseline) + kernel
典型应用	阶跃响应 (>10 ns IIR 尾)	方波拖尾	方波拖尾、复杂波形	高频成分、宽带瞬态
工作点	敏感点 ($d/d\Phi$)	敏感点	敏感点	任意 (但越敏感越好)
代码	<code>experiments/cryoscope.py</code> + <code>reconstruction/cryoscope.py</code>	<code>experiments/delay_ramsey.py</code> + <code>reconstruction/delay_ramsey.py</code>	<code>experiments/pi_pulse_comp.py</code> + <code>reconstruction/pi_pulse_comp.py</code>	<code>experiments/transient.py</code> + <code>reconstruction/transient.py</code>

Notebook §10.2.3 给出的实证比较 (阶跃响应同一组 qubit/失真/step_amp 上):

方法	RMSE (Φ)	上升沿 (ns)	采样点	备注
真实值	—	~3 ns	—	<code>dist.step_response()</code>
Cryoscope	~ 1×10	与真实接近	60+ trunc	相位精度高、需要
Transient (Wiener)	~ 5×10	略宽	70+ scan	时间分辨率好、噪声敏感

最佳实践: Cryoscope 处理 >10 ns 的 IIR-type 尾, Transient 处理 <10 ns 的 FIR-type 快瞬态——二者互补。

主线 B • Qubit 频率标定

目标: 建立 f 在 (Φ, V) 参数空间上的“刻度”。两个标定器类、四种 method、一个调度器。

2.1 标定层概览

频率主线分测量 (read-only) 与调谐 (write/feedback) 两个职责清晰的类, 由 v2.5 重构落定。FluxResponseCalibration 给宽视野 $f(\Phi)$ 曲线; FrequencyMeasurement 给单点 f ; SinglePointFrequencyCalibration 用闭环把 f 推到目标值, 内部组合一个 FrequencyMeasurement 实例完成每步测频。

sqc.calibration

CalibrationTable
(base.py:19)

Calibration (ABC)
(base.py:167)

CalibrationScheduler
(scheduler.py:19)

evaluate(x)
inverse(y)

FluxResponseCal
(frequency.py)
"ramsey"
"transient"*

f(Φ) 曲线

* Track B 1.2 待实现

FrequencyMeasurement
(frequency.py, v2.5)
"ramsey"
"transient"

单点 f 测量
(read-only)
.measure(flux) -> float
.calibrate() -> CalibrationTable

Track B 1.2 已接通

SinglePointFreqCal
(frequency.py)
"closed_loop"
secant

bisection

+ 内部 FrequencyMeasurement
(闭环调谐)

为未来调谐策略预留 method 字段

(gradient / feedforward 等)

WaveformCal
(waveform.py)
"transfer_function"
"predistortion"

注: FluxResponseCalibration / FrequencyMeasurement / SinglePointFrequencyCalibration 是频率主线的核心; WaveformCalibration / PredistortionDesigner 形式上也属于 calibration 模块, 但属于主线 C (§3.3 详述)。

测量与调谐的职责分离——这是 v2.5 重构的核心动机:

维度	FrequencyMeasurement	SinglePointFrequencyCalibration
物理目标	表征 (read f at given flux)	调谐 (drive f $\rightarrow$ f_target)
反馈回路	无	有 (secant/bisection on $r(V) = 0$)
必填参数	qubit	qubit, f_target, V_a, V_b
调用次数	1 $\times$	N $\times$ (每个闭环迭代调一次 measurement)
接口形态	.measure(flux) -> float (子例程) + .calibrate() -> CalibrationTable (独立任务)	.calibrate() -> CalibrationTable (带 V_opt + history)
内部依赖	模块级 _fit_ramsey_frequency 或 _measure_frequency_transient	持有 self._meas: FrequencyMeasurement 实例 (dual-sweep ramsey 或 transient)

CalibrationScheduler 是一个轻量的 “控制室”:

- **注册** (scheduler.py:61-102): 维护任务名 $\rightarrow$ Calibration 类的映射, 及依赖关系。
- **运行** (scheduler.py:108-172): run(name) 检查依赖 $\rightarrow$ 自动注入依赖结果 (如 flux_response_ramsey 的 inputs 自动作为 closed_loop 的 V_a/V_b bracket) $\rightarrow$ 执行。
- **DAG 自动化** (scheduler.py:261-316): check_state / maintain / auto_calibrate 是 Kelly 2018 风格的 stub, 留给未来。

预注册任务列表 (scheduler.py:82-102):

任务名	类	依赖
flux_response_ramsey	FluxResponseCalibration(method="ramsey")	
frequency_measurement	FrequencyMeasurement (method= "ramsey" 默认 / "transient")	—

任务名	类	依赖
frequency_closed_loop	SinglePointFrequencyCalibration	FluxResponseRamsey
waveform_transfer_function	WaveformCalibration	FluxResponseRamsey
waveform_predistortion	WaveformCalibration	FluxResponseRamsey

2.2 共用基石: Ramsey FFT 频率拟合

[sqc/calibration/frequency.py:28](#) `_fit_ramsey_frequency()` ——跑一组 Ramsey 扫，FFT 找 detuning 峰:

```
def _fit_ramsey_frequency(qubit, omega_d, tau_list, t_rabi, t_global, flux=0.0):
    # 1. 把 qubit 钉在指定 flux
    Phi = FluxSignal(type=1 if flux != 0.0 else 0, t_list=t_sig,
                     amplitude=float(flux))
    qubit.qubit_in_mag(Phi, frame=1, omega_d=omega_d)

    # 2. 扫 tau 跑 Ramsey
    for i, tau in enumerate(tau_list):
        ctrl = create_ramsey_pulse(t_rabi, tau, omega_d=omega_d,
                                   phase1=0.0, phase2=0.0, qubit=qubit)
        ctrl.t_list = ctrl.t_list - t_rabi[-1]
        H = (QobjEvo(qubit.H_list, tlist=qubit.mag_signal.t_list, order=1)
             + QobjEvo(ctrl.hamiltonian, tlist=ctrl.t_list, order=1))
        result = mesolve(H, qubit.state, t_global, [], e_ops=[psi_e * psi_e.dag()])
        p_e_vals[i] = float(result.expect[0][-1])

    # 3. FFT + 二次插值精确找峰
    p_centered = p_e_vals - np.mean(p_e_vals)
    fft_vals = np.abs(rfft(p_centered, n=2048))
    freqs = rfftfreq(2048, d=dt_val)
    peak_idx = int(np.argmax(fft_vals))

    # 子-bin 二次插值 (关键: 把分辨率从 1/n_fft 提升到连续)
    y1, y2, y3 = fft_vals[peak_idx-1], fft_vals[peak_idx], fft_vals[peak_idx+1]
    denom = 2 * (y1 + y3 - 2 * y2)
    delta_bin = (y1 - y3) / denom
    detuning_hz = (peak_idx + delta_bin) * df_bin
    return float(omega_d) + 2 * np.pi * detuning_hz
```

子-bin 二次插值 ([frequency.py:103-117](#)) 是这个标定器的核心——把“找峰的分辨率”从 $1/T_{\text{total}}$ 提到连续值，FFT bin 数 2048 时 ~kHz 级。

2.3 FluxResponseCalibration 一扫 Φ 拟 $f(\Phi)$

[sqc/calibration/frequency.py:125](#) `FluxResponseCalibration`:

```
@dataclass
class FluxResponseCalibration(Calibration):
    qubit: object
    method: Literal["ramsey", "transient"] = "ramsey"
    h_list: np.ndarray | None = None # default linspace(-0.03, 0.03, 51)
    tau: float = 100.0
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
```

`_calibrate_ramsey()` ([frequency.py:170-192](#)):

1. 对每个 $h \in h_list$ 调用 `_fit_ramsey_frequency(qubit, omega_d, tau_list, t_rabi, t_global, flux=h)`
2. 输出 `CalibrationTable(kind="f_phi", inputs=h_list, outputs=f01_list)`

`_calibrate_transient()` 是 stub ——Track B 1.2 未实现 (在 sqc 框架中保留接口位置，实际算法待补)。

用途:

- 给 closed-loop 标定提供 V_a , V_b bracket (scheduler 自动注入)。
- 作为 Cryoscope/Delay Ramsey 标定的“宽视野” sanity check。

2.4 FrequencyMeasurement —单点 f 测量

v2.5 重构说明: 旧版 SinglePointFrequencyCalibration 把“测频”和“闭环调谐”两件事都塞在一个 method 字段里 ("ramsey"/"closed_loop"/"transient"), 导致 closed_loop 既要管自己的搜索逻辑又要重复 dispatch 测频逻辑, 参数列表里又混杂着只对部分 method 有意义的字段。v2.5 拆分后: 测频归 FrequencyMeasurement, 调谐归 SinglePointFrequencyCalibration (§2.5), 闭环里“内部组合”一个 FrequencyMeasurement 实例完成每步迭代——这是经典 strategy + composition 范式, 未来加新的调谐方法 (gradient、feedforward 等) 只需新加 case, 不动测频。

[sqc/calibration/frequency.py:317](#) FrequencyMeasurement 是单点 f 测量类, 提供两种测量方式:

method	物理原理	时间代价	何时用
"ramsey"	Ramsey -sweep + FFT 提失谐峰 (含单/双扫两个子模式)	$\text{len}(\text{tau_list}) \times \text{mesolve}$ (默认 $\sim 80\times$)	高精度独立测频; 闭环内层 (双扫)
"transient"	=0 正交 Ramsey + 核函数灵敏度 $G_{\perp}$	$2 \times \text{mesolve} + 1 \times \text{kernel}$	弱信号近似下的高速测频 (闭环内层加速)

两种接口形态:

- `.measure(flux=None) -> float`: 直接返回 f (rad·GHz), 作为子例程被闭环反复调用。
- `.calibrate() -> CalibrationTable`: 标准 Calibration 接口, 包装单点结果供 scheduler 注册和后续 evaluate/inverse 调用。

数据类签名 ([frequency.py:317-378](#)):

```
@dataclass
class FrequencyMeasurement(Calibration):
    qubit: object
    method: Literal["ramsey", "transient"] = "ramsey"
    flux: float = 0.0  # 默认测 sweet spot

    # Ramsey-specific (method="transient" 时只用 t_rabi / t_global)
    tau_list: np.ndarray | None = None  # default make_time(0, 200)
    t_rabi: np.ndarray = field(default_factory=lambda: CONFIG.pulse.t_rabi.copy())
    t_global: np.ndarray | None = None  # default CONFIG.pulse.t_global
    f_artificial: float | None = 0.1  # 单扫 =float, 双扫 =None
```

2.4.1 method="ramsey" ——Ramsey FFT 双模

底层调用 [frequency.py:98-194](#) `_fit_ramsey_frequency`, 通过 `f_artificial` 切换单/双扫两种子模式:

单扫 (`f_artificial = 0.1 GHz`, 默认):

第二个 $/2$ 脉冲带相位斜坡 $\text{phase2} = 2 \cdot f_a$, 在旋转坐标系中相当于额外加了人工失谐 f_a 。FFT 测得:

$$f_{\text{meas}} = |\Delta + f_a| = \Delta + f_a \quad (\text{要求 } f_a > |\Delta|_{\text{max}})$$

$\Delta = f_{\text{meas}} - f_a$ 。1× -sweep 即可, 但调用者必须保证 $f_a > |f_q - \omega_d|_{\text{max}}$ 。sweet spot 附近窄 flux 扫描时合适。

双扫 (`f_artificial = None`):

跑 ± 0.05 GHz 两轮 -sweep, 符号失谐由恒等式:

$$\Delta = \frac{f_+^2 - f_-^2}{4f_a}, \quad f_a = 0.05 \text{ GHz}$$

2× 代价但 $|\Delta|$ 任意大、带符号。这是闭环必用模式——搜索过程中 V 可能远离 sweet spot, $|\Delta|$ 可达 GHz 量级, 固定 f_a 的单扫会反推出错符号。

FFT 找峰底层 ([frequency.py:29-53](#) `_fft_peak`): `rfft` (zero-pad 到 `n_fft=2048`) $\rightarrow$ `argmax` $\rightarrow$ 三点抛物线子-bin 插值:

$$\delta_{\text{bin}} = \frac{y_1 - y_3}{2(y_1 + y_3 - 2y_2)}, \quad f_{\text{meas}} = (k^* + \delta_{\text{bin}}) \cdot df_{\text{bin}}$$

把分辨率从 $1/T_{\text{max}} \sim 5$ MHz 提升到 ~ 250 kHz。低对比度保护: $\text{ptp}(\mathbf{p}_e) < 0.01$ 直接返回 None, 上层降级返回原 ω_d 。

2.4.2 method="transient" ——正交 Ramsey 差分 + 核函数 G_{α}

底层调用 `frequency.py:201-285 _measure_frequency_transient` (v2.5 起从原 `SinglePointFrequencyCalibration` 的方法提升为模块级函数, 让 `FrequencyMeasurement(method="transient")` 和闭环内层都能共用)。

构造两个正交 $\omega=0$ Ramsey: $\text{ctrl}_x = (R_y, R_x)$ 与 $\text{ctrl}_{mx} = (R_y, R_{-x})$, 跑 mesolve 得 p_x, p_{mx} 。差分:

$$p_{\text{diff}} = \frac{1}{2}(p_x - p_{-x})$$

理论上 (`_sensing theory.md` § 瞬态磁场协议) 弱信号近似:

$$p_{\text{diff}} \approx G_{\alpha} \cdot \Delta\omega, \quad G_{\alpha} = \int k(t) dt$$

所以 $\text{delta_omega} = p_{\text{diff}} / G_{\alpha}$, 最终 $f_q = \omega_d + \text{delta_omega}$ 。

```
# sqc/calibration/frequency.py:201-285 (节选)
ctrl_x = create_ramsey_pulse(t_rabi, tau=0.0, omega_d=omega_d,
                             phase1=np.pi/2, phase2=0.0, qubit=qubit)
ctrl_mx = create_ramsey_pulse(t_rabi, tau=0.0, omega_d=omega_d,
                              phase1=np.pi/2, phase2=np.pi, qubit=qubit)

# ... mesolve 两次得 p_x, p_mx
p_diff = (p_x - p_mx) / 2.0

# kernel sensitivity G_ = k(t) dt
ctrl_x.get_kernel(qubit)
G_alpha = float(np.trapezoid(ctrl_x.kernel, ctrl_x.t_samples))

# restore qubit state after get_kernel side effects
qubit.qubit_in_mag(Phi, frame=1, omega_d=omega_d)

if abs(G_alpha) < 1e-15:
    return float(omega_d)
delta_omega = p_diff / G_alpha
return float(omega_d + delta_omega)
```

优势 / 限制:

- 每次测量只跑 $2 \times \text{mesolve} + 1 \times \text{kernel}$ (kernel 可后续 caching), 比 Ramsey FFT 快 $\text{len}(\text{tau_list})/2 \sim 25\text{-}50\times$ 。
- 但依赖弱信号线性近似, $|\Delta \cdot \text{tau}_R| > 1$ 时近似失效。闭环搜索远离 sweet spot 时不推荐用 transient 内层。

2.4.3 调用示例

独立单点测频:

```
from sqc.calibration.frequency import FrequencyMeasurement

# 方法 A: Ramsey 单扫 → 标准 CalibrationTable
m = FrequencyMeasurement(qubit=q, method="ramsey")
table = m.calibrate()
print(table.outputs[0]) # f_q (rad/GHz)

# 方法 B: Ramsey 双扫 → 直接拿 float
m = FrequencyMeasurement(qubit=q, method="ramsey", f_artificial=None)
f_q = m.measure(flux=0.025) # 在指定 flux 测, 不打包

# 方法 C: 瞬态测频 (高速, 弱信号近似)
m = FrequencyMeasurement(qubit=q, method="transient", flux=0.0)
table = m.calibrate()
```

作为子例程被闭环调用 (§2.5 详述): 闭环内部在 `__post_init__` 一次性构造 `self._meas = FrequencyMeasurement(...)`, 每次迭代调 `self._meas.measure(V_n)`。

2.4.4 通过 Scheduler 注册

`frequency_measurement` 是 v2.5 注册的统一入口, 可独立运行或被其他任务依赖:

```
sch = CalibrationScheduler()
sch.register_defaults()
# 默认 method="ramsey"; 用 kwargs 切换
table = sch.run("frequency_measurement", qubit=q, method="transient", flux=0.005)
```

2.4.5 约束与陷阱

约束	出处	说明
单扫模式要求 $ \Delta < f_a$	frequency.py:118-127	否则 FFT 取绝对值会反推出错符号; 闭环远离 sweet spot 必须用双扫 (<code>f_artificial=None</code>)
FFT 分辨率 $1/T_{\max}$	<code>_fft_peak</code>	默认 <code>tau_list=make_time(0,200) → df_bin=5 MHz</code> , 子-bin 插值后 ~250 kHz
低对比度返回 <code>omega_d</code>	frequency.py:34-35	振幅 <code>ptp(p_e) < 0.01</code> 时上层会 “假收敛” ——sweet spot 紧贴 <code>_d</code> 时易触发
Transient 假设线性	frequency.py:282	$ \Delta \cdot \tau_R > 1$ 时偏差大; 仅在弱信号近似有效区间用
Transient 改 qubit 状态	frequency.py:281	<code>get_kernel</code> 有副作用, 函数末尾 <code>qubit_in_mag(Phi)</code> 恢复; 复用 qubit 实例时要意识到这层
Ramsey + transient 都先调 <code>qubit_in_mag</code> 设 <code>flux</code>	frequency.py:158, 220	复用 qubit 实例做多次测量时需注意上下文切换开销

2.5 SinglePointFrequencyCalibration —单点 f 调谐

[sqc/calibration/frequency.py:454](#) `SinglePointFrequencyCalibration` 是单点频率调谐类 (v2.5 起瘦身为只含调谐方法)。把 “找 V 使 $f(V) = f_{\text{target}}$ ” 当成一个单变量根求解问题:

$$r(V) := f_Q(V) - f_{\text{target}} = 0$$

数据类签名 ([frequency.py:454-516](#)):

```
@dataclass
class SinglePointFrequencyCalibration(Calibration):
    qubit: object
    method: Literal["closed_loop"] = "closed_loop"  # 仅一个值, 为未来调谐策略预留

    # closed_loop 必填
    f_target: float | None = None  # 目标频率 (rad.GHz)
    V_a: float | None = None  # bracketing 下界
    V_b: float | None = None  # bracketing 上界

    # 闭环参数
    epsilon_f: float = 1e-4  # 收敛容差 (rad.GHz)
    max_iter: int = 20
    measure_method: Literal["ramsey", "transient"] = "ramsey"
    bracket_tightening: bool = True
    step_method: Literal["secant", "bisection"] = "secant"

    # 内部 FrequencyMeasurement 转发参数
    tau_list, t_rabi, t_global: ...
```

关键设计: `__post_init__` 中一次性构造 `self._meas = FrequencyMeasurement(..., f_artificial=None)` (dual-sweep 强制开启, 保证搜索过程中即使 V 远离 sweet spot 也能正确测频)。闭环算法的每步迭代调用 `self._meas.measure(V_n)`——这就是“调谐组合测量”的具象。

method 字段保留 Literal["closed_loop"] 看似冗余, 但是给未来扩展 (gradient descent、feedforward、ML-driven 等) 留好的扩展点: `calibrate()` 的 `match self.method` 派发结构已在那里, 只需补 case "gradient": 即可上新方法, 无需改类签名。

2.5.1 顶层入口

`frequency.py:524-552` `_calibrate_closed_loop()`:

```
def _calibrate_closed_loop(self) -> CalibrationTable:
    if self.f_target is None:
        raise ValueError("f_target is required for closed_loop method.")
    if self.V_a is None or self.V_b is None:
        raise ValueError("V_a and V_b are required ...")

    V_lo, V_hi = min(self.V_a, self.V_b), max(self.V_a, self.V_b)
    if self.step_method == "bisection":
        return self._closed_loop_bisection(...)
    else:
        return self._closed_loop_secant(...)
```

设计契约: 闭环不负责找 bracket——找 bracket 是 `FluxResponseCalibration` 的工作 (粗扫 $f(\Phi)$ 表, 定位 `f_target` 所在电压窗)。这是一个明确的工作流约束: 粗标 → 闭环细调。

2.5.2 Secant 法 (默认 `step_method="secant"`)

`frequency.py:555-601`。割线公式:

$$V_{n+1} = V_n - r_n \cdot \frac{V_n - V_{n-1}}{r_n - r_{n-1}}$$

初始化代价: 在主循环前已花 $2 \times$ `self._meas.measure(...)`——中点起步 $V_n = (V_{lo} + V_{hi})/2$ 、前一点取 $V_{prev} = V_{lo}$ 。中点起步而非端点的三个考量: 中点 r 值小, 第一步 V_{next} 容易落 bracket 内 端点是粗标边界精度低 多花 1 次测量换收敛少 1 步, 净开销持平。

每轮三步:

1. **Secant 公式 + 两层兜底:** 分母 $|r_n - r_{prev}| < 1e-15$ 退回中点 (近平台兜底); 外推越界 $[V_{lo}, V_{hi}]$ 退回中点 (且不更新 V_{prev}/r_{prev} , 等于白跑一步换二分稳健性)。
2. **测频:** $r_n = \text{self._meas.measure}(V_n) - f_{\text{target}}$ ——每个 iter 真正的计算瓶颈, $\sim 2 \times N_{\text{mesolve}}$ $100 \times$ mesolve (双扫 ramsey) 或 $2 \times$ mesolve (transient)。
3. **Bracket 收紧 Regula falsi** (`bracket_tightening=True`, 默认): $r_n > 0$ $V_{hi} = V_n$, 否则 $V_{lo} = V_n$ 。这让纯 secant 等价于 Illinois 变体——收敛阶 1.618 (超线性) + bisection 级别的不发散保证。

`bracket_tightening=False` 用于诊断纯 secant 行为 (远离根可能发散)。

每轮 history 记 {iter, V, f, residual}。

2.5.3 Bisection 法 (`step_method="bisection"`)

`frequency.py:604-660`。每次取中点缩半区间:

$$V_n = \frac{1}{2}(V_{lo} + V_{hi}), \quad \text{bracket width} \sim \frac{V_b - V_a}{2^n}$$

唯一新增能力 vs secant: 偶对称 $f(\Phi)$ 自动 split——若 $r(V_{lo}) \cdot r(V_{hi}) > 0$ (甜点两侧 f 同向衰减导致两端同号), 取中点 V_{mid0} 试探, 根据三段符号判断目标在哪半边 (`frequency.py:619-633`)。若三点同号 整区间无根 `raise RuntimeError`, 要求重新跑 `FluxResponseCalibration` 取更宽窗。

每轮 history 额外记 `bracket_width` 字段, 便于绘制对数衰减直线。

收敛速度: bisection 线性 (每步 $\times 0.5$), 约 $\log((V_b - V_a)/\epsilon)$ 10-15 步达到 $\epsilon = 1e-4$; secant + bracket tightening 超线性, 1-3 步。所以 secant 是生产默认, bisection 是诊断/演示模式 (log-linear 曲线适合论文图)。

2.5.4 内嵌测频策略

闭环每步迭代调 `self._meas.measure(V)`, 由 `FrequencyMeasurement(method=measure_method, f_artificial=None)` 完成。两个策略:

measure_method	闭环单步代价	适用范围	总迭代数 (典型)	总 mesolve 数
"ramsey" (默认, 强制双扫)	$\sim 2 \times N_{\text{mesolve}} \times 100 \times$	任意 $ \Delta $, 鲁棒	1-3 (secant) / 10-15 (bisection)	200-400 / 1000-1500
"transient"	$2 \times \text{mesolve} + 1 \times \text{kernel}$	弱信号近似有效区	同上	4-6 / 20-30

闭环里强制 `f_artificial=None` 的原因: 搜索过程中 V 可能远离 sweet spot, $|\Delta|$ 可达 GHz 量级, 单扫模式的固定 $f_a=0.1$ GHz 会反推出错符号。

2.5.5 输出 CalibrationTable 结构

`_build_result (frequency.py:662-684):`

```
CalibrationTable(
    name="frequency_closed_loop",
    kind="f01",
    inputs=np.array([V_opt]),
    outputs=np.array([f_target + residual]),
    fit_params={
        "method": "closed_loop",
        "step_method": "secant" | "bisection",
        "measure_method": "ramsey" | "transient",
        "f_target": float,
        "V_opt": float,
        "n_iter": int,
        "residual": float,
        "converged": bool,
        "history": list[dict],
    },
)
```

abs(residual) <= epsilon
每步 {iter, V, f, residual, [bracket_width]}

2.5.6 Notebook 调用示例

完整 pipeline (`Simulation_sqc.ipynb` §9 + `closed_loop_calibration_test.ipynb`):

Step 1: 粗标定 $f(\Phi)$ 给 bracket:

```
qubit = TransmonQubit(EC=0.2*2*np.pi, EJ=10.0*2*np.pi,
                      T1=100e3, T2=50e3, flux=0.0, n_levels=2)
flux_cal = FluxResponseCalibration(qubit=qubit, method="ramsey",
                                   h_list=np.linspace(-0.03, 0.03, 15), tau=100.0)
flux_table = flux_cal.calibrate()
V_a, V_b = float(np.min(flux_table.inputs)), float(np.max(flux_table.inputs))

# 选个目标:  $\Phi=0.025$  处的频率
f_interp = interp1d(flux_table.inputs, flux_table.outputs, kind="cubic")
f_target = float(f_interp(0.025))
```

Step 2: 闭环调谐 (两种内层测量策略 + bisection):

```
# 方法 A: Ramsey FFT 内层 + Bisection (高精度, 鲁棒)
cal_ramsey = SinglePointFrequencyCalibration(
    qubit=qubit,
    measure_method="ramsey",
    step_method="bisection",
    tau_list=CONFIG.pulse.make_time(0, 500),
    f_target=f_target,
    epsilon_f=1e-4 * 2 * np.pi,
    V_a=V_a, V_b=V_b, max_iter=20,
)
```

method="closed_loop" 是默认值, 可省
长 提升 FFT 分辨率

```

res_r = cal_ramsey.calibrate()

# 方法 B: Transient kernel 内层 + Bisection (高速, 弱信号近似)
cal_transient = SinglePointFrequencyCalibration(
    qubit=qubit2,
    measure_method="transient",
    step_method="bisection",
    f_target=f_target,
    epsilon_f=1e-4 * 2 * np.pi,
    V_a=V_a, V_b=V_b, max_iter=20,
)
res_t = cal_transient.calibrate()

```

Step 3: 收敛对比可视化展示 6 张子图:

1. (a) $f(\Phi)$ 曲线 + target 标注
2. (b) V vs iter 收敛轨迹
3. (c) f vs iter
4. (d) $|\text{residual}|$ log 衰减
5. (e) bracket width log 衰减 (验证 1/2 理论线)
6. (f) Summary 表 (迭代次数 + 单次测量代价 + 总测量数)

关键观察: transient kernel 每次迭代仅需 2 次单点测量, 比 Ramsey FFT 快 $\text{len}(\text{tau_list})/2$ 25-50×, 精度相当 (在弱信号近似有效区间内)。

2.5.7 通过 Scheduler 注册 (依赖注入)

frequency_closed_loop 任务依赖 flux_response_ramsey——scheduler 自动把 flux 扫描表的 inputs 注入为 V_a/V_b bracket (`scheduler.py:174-192`):

```

sch = CalibrationScheduler()
sch.register_defaults()
sch.run("flux_response_ramsey", qubit=q)
sch.run("frequency_closed_loop",
        qubit=q, f_target=2*np.pi*5.20)

```

必须先跑这个
自动注入 V_a, V_b
只需提供 f_target

2.5.8 未来扩展的预留口

method: Literal["closed_loop"] 字段在 v2.5 保留有意为之——calibrate() 中的 match self.method 派发结构已就位, 扩展新调谐策略只需:

```

# 例如在 __post_init__ 加 measurement strategy 选择, 在 calibrate() 加 case:
match self.method:
    case "closed_loop":
        return self._calibrate_closed_loop()
    case "gradient":
        return self._calibrate_gradient()
    case "feedforward":
        return self._calibrate_feedforward()

```

新增
新增

类签名、Scheduler 注册名、文档结构都不动。

2.5.9 约束与陷阱

约束	出处	说明
f_target / V_a / V_b 必填	frequency.py:533-541	否则 ValueError
Bisection 起始两端同号	frequency.py:618-633	自动用中点试探切分 bracket; 若三点同号则 raise——重跑 FluxResponse 取更宽窗
Secant 可能跳出 bracket	frequency.py:570-575	设了 fallback 到 midpoint 兜底

约束	出处	说明
闭环 ramsey 内层强制双扫	frequency.py:516	f_artificial=None 写死在 __post_init__, 避免远离 sweet spot 反推错符号
闭环 transient 内层风险	frequency.py:282	弱信号近似在远离根的搜索点失效; 闭环起步阶段尤其要警惕
Bracket 跨越多个 sweet spot	—	f(Φ) = cos(Φ) 周期函数, 若 V_a/V_b 跨越多个周期, secant 会飘移; 先用 FluxResponse inverse(f_target) 给出粗 V_opt 后再 ±0.005 Φ 收紧 V_a/V_b
收敛容差被 FFT 分辨率限制	—	epsilon_f < 1/T_max 时 ramsey 内层抖动会让 secant 看似不收敛; tau_long=500 ns → ~1 MHz 极限
Kernel 必须在 qubit 当前 flux 下重新算	frequency.py:570-576	否则 G_ 不对

主线 C • 预失真

目标: 测出控制线的传递函数 $H()$, 设计逆滤波器, 让 AWG → 片上波形误差最小。

3.1 失真模型层

[sqc/hardware/distortion.py](#) 定义了 5 个 LTI 失真模型, 全部实现 DistortionModel ABC ([distortion.py:31](#)) 的 4 个接口: apply(), step_response(), impulse_response(), frequency_response()。

3.1.1 SingleExponentialDistortion — 单指数尾

[distortion.py:105](#)。这是 Cryoscope 标定中最常见的 IIR-type 失真:

$$h(t) = (1 - \alpha) \delta(t) + \frac{\alpha}{\tau} e^{-t/\tau} \Theta(t)$$

$$H(s) = \frac{1 + s\tau(1 - \alpha)}{1 + s\tau}, \quad H(\omega) = (1 - \alpha) + \frac{\alpha}{1 + j\omega\tau}$$

$$s(t) = 1 - \alpha e^{-t/\tau} \quad (\text{step response})$$

代码 ([distortion.py:152-179](#)): 用 bilinear transform 把连续时间 $H(s)$ 离散化成一阶 IIR (b, a), 再用 scipy lfilter 应用。

3.1.2 MultiExponentialDistortion — K 个指数并联

[distortion.py:187](#):

$$H(\omega) = (1 - \sum_k \alpha_k) + \sum_k \frac{\alpha_k}{1 + j\omega\tau_k}, \quad s(t) = 1 - \sum_k \alpha_k e^{-t/\tau_k}$$

apply() ([distortion.py:218-237](#)): DC gain 直传 + K 个并联 IIR 累加。

3.1.3 FIR / IIR / Custom

类	描述	应用
FIRDistortion	$y[n] = \sum b[k] \cdot x[n-k]$	短脉冲响应
IIRDistortion	$y[n] = \sum b[k] \cdot x[n-k] - \sum a[k] \cdot y[n-k]$	反馈系统
CustomTransferDistortion	用户提供 $H()$ on grid	从 Cryoscope/Transient 测得的非参数 H

apply_to_waveform() ([distortion.py:84-97](#)) 是统一适配器: 接收 Waveform, 调底层 apply(samples, dt), 返回新 Waveform。

3.2 控制线封装 ControllLine

`sqc/hardware/control_line.py:19` 把 `DistortionModel` 装到一条物理线上:

```
@dataclass
class ControllLine:
    name: str # "Z0"
    kind: Literal["xy", "z", "readout"]
    source: str # "AWGO:CHO"
    target: str # "Q0"
    transfer_function: Optional["DistortionModel"] = None
    # 物理参数 (可选)
    impedance: float = 50.0
    attenuation_db: float = 20.0
    delay: float = 0.0
```

两个核心方法:

- `apply(awg_waveform)` (`control_line.py:65-99`) —— 正向模型: AWG → 片上。如果有 `transfer_function`, 调用 `apply_to_waveform()` 加失真; 如果有 `delay > 0`, 做样本级时移。
- `predistort(target, designer)` (`control_line.py:101-120`) —— 反向: 目标片上 → 所需 AWG, 委托 `PredistortionDesigner`。

3.3 标定层 WaveformCalibration + PredistortionDesigner

3.3.1 WaveformCalibration —— 统一入口

`sqc/calibration/waveform.py:27` `WaveformCalibration` 有两种 method:

A. `method="transfer_function"` (`waveform.py:103-119`): 1. 用 `distortion.step_response(t)` 直接得到阶跃响应
2. 用 `_fit_step_response()` 拟合成参数化模型 3. 返回 `CalibrationTable(kind="transfer_function", inputs=t, outputs=step, fit_params=...)`

`fit_type` 四种:

fit_type	拟合函数	物理模型
"single_exp"	$s(t) = 1 - \text{amp} \cdot \exp(-t/\tau)$	SingleExponentialDistortion
"multi_exp"	$s(t) = 1 - \sum \text{amp}_k \cdot \exp(-t/\tau_k)$	MultiExponentialDistortion
"fir"	impulse h ds/dt 截 N taps	FIRDistortion
"iir"	1 阶 IIR 双线性	IIRDistortion

`_fit_multi_exp` (`waveform.py:163-214`): 先串行剥离 **K 次单指数** (每次拟合后从残差里减掉), 再一次性 `curve_fit` 联合优化所有 2K 参数。这种 “greedy 初始化 + 联合精修” 比直接 K-参数初始化更稳。

`to_distortion_model()` (`waveform.py:239-269`): 把标定结果包成对应的 `DistortionModel` 子类。

B. `method="predistortion"` (`waveform.py:275-296`): 把 `transfer_model` 喂给 `PredistortionDesigner`, 得到逆滤波器, 包成 `CalibrationTable(kind="predistortion", fit_params={"inverse_model": ...})`。

3.3.2 PredistortionDesigner —— 三种逆滤波器设计

`sqc/calibration/waveform.py:304` `PredistortionDesigner`:

```
@dataclass
class PredistortionDesigner:
    method: Literal["auto", "fir_inverse", "iir_inverse", "frequency_inverse"] = "auto"
    n_taps: int = 64
    regularization: float = 1e-4
```

`design(transfer_model, dt)` (`waveform.py:326-351`) 根据 `method` 派遣到三条路径:

A. `iir_inverse` (指数失真的解析逆) (`waveform.py:360-397`):

对 `SingleExponentialDistortion(amp, tau)`: 原始 $H(s) = \frac{1 + s\tau(1 - \alpha)}{1 + s\tau}$, 逆为:

$$H^{-1}(s) = \frac{1 + s\tau}{1 + s\tau(1 - \alpha)}$$

```
# sqc/calibration/waveform.py:373-383
def _single_exp_to_iir_inverse(model, dt):
    amp, tau = model.amplitude, model.tau
    b_cont = [tau, 1.0]
    a_cont = [tau * (1.0 - amp), 1.0]
    b, a = bilinear(b_cont, a_cont, fs=1.0 / dt)
    return IIRDistortion(b_coeffs=b, a_coeffs=a)
```

对 MultiExponentialDistortion: 直接用频域 Wiener 逆 (waveform.py:385-397), 因为多个并联指数的 IIR 解析逆是高阶有理函数, 不稳定。

B. fir_inverse (FIR 频域 Wiener 逆) (waveform.py:401-418):

构造 输入 → 跑正向 → 得经验 impulse response → FFT → Wiener 逆 → IFFT 截 N taps:

```
n_fft = 4096
imp = np.zeros(n_fft); imp[0] = 1.0 / dt
h_fwd = transfer_model.apply(imp, dt)
H_emp = np.fft.fft(h_fwd)
H_inv = np.conj(H_emp) / (np.abs(H_emp) ** 2 + self.regularization ** 2)
h_inv_full = np.fft.ifft(H_inv).real
h_trunc = h_inv_full[:self.n_taps]
return FIRDistortion(taps=h_trunc)
```

C. frequency_inverse (直接频域) (waveform.py:422-435):

调 transfer_model.frequency_response(omega_grid) 拿 $H()$, Wiener 逆后包成 CustomTransferDistortion。

"auto" 选择规则 (waveform.py:353-356): 类名含 "Exponential" 走 iir_inverse, 否则走 frequency_inverse。

3.3.3 稳定性检查

waveform.py:459-465 check_pole_stability(b, a):

所有 $|p_i| < 1$ (其中 p_i 是 $a(z)$ 的根) $\iff$ IIR 稳定

```
@staticmethod
def check_pole_stability(b_coeffs, a_coeffs) -> bool:
    roots = np.roots(a_coeffs)
    return bool(np.all(np.abs(roots) < 1.0 - 1e-10))
```

——用于 IIR 解析逆设计后做 sanity check。

3.4 端到端 workflow PredistortionValidationWorkflow

sqc/workflows/predistortion_validation.py:28 把 §3.1-§3.3 串成 6 步:

```
@dataclass
class PredistortionValidationWorkflow(Workflow):
    target_waveform: Waveform
    true_distortion: object # DistortionModel
    designer: Optional[object] = None # PredistortionDesigner, 默认 auto
    control_line_params: dict = field(default_factory=dict)
```

run() 流程 (predistortion_validation.py:58-118):

1. 建控制线 —— ControllLine(name, kind, ..., transfer_function=true_distortion)
2. 无补偿测量 —— on_chip_uncorrected = line.apply(target_waveform)
3. 标定 $H()$ —— WaveformCalibration(distortion=true_distortion, method="simulation", fit_type=auto_inferred).calibrate()
4. 设计逆 —— inverse_model = designer.design(measured_model, dt)
5. 预失真 —— awg_predistorted = inverse_model.apply_to_waveform(target)
6. 再次正向 —— on_chip_corrected = line.apply(awg_predistorted)
7. 指标对比 —— RMSE before/after + settling time before/after

`_infer_fit_type` (`predistortion_validation.py:139-150`): 根据真实失真类名自动选 `fit_type`.

指标计算 (`predistortion_validation.py:156-195`):

```
@staticmethod
def _rmse(measured, target):
    return float(np.sqrt(np.mean((measured.samples - target.samples) ** 2)))

@staticmethod
def _settling_time(w, target, tolerance=0.001):
    error = np.abs(w.samples - target.samples)
    bad = np.where(error > tolerance)[0]
    if len(bad) == 0: return 0.0
    return float(w.t_list[bad[-1]])
```

返回 dict 含: target, on_chip_uncorrected, on_chip_corrected, awg_predistorted, inverse_model, measured_model, metrics.

3.5 Z-线交叉串扰 ZCrosstalkWorkflow

`sqc/workflows/z_crosstalk.py:25` ZCrosstalkWorkflow ——多 qubit 场景下, Z 控制线之间的电磁串扰使得“驱动 QA 时 QB 也感受到寄生磁通”。

物理: 定义交叉传递矩阵 $H_BA() = \Phi_B() / V_A()$ 。用 transient sensing 在 QB 上反演 $_B(t)$, FFT 出 H_BA 。

`run()` 流程 (`z_crosstalk.py:81-142`):

1. 应用真实传递矩阵 ——`on_chip_fluxes = true_transfer_matrix.apply({QA: V_A})`, 得到 `phi_A`, `phi_B_true`
2. QB 上 transient sensing (`z_crosstalk.py:148-180`):
 - 把 QB 移到 sensitivity 最大点 (`_make_qubit_at_optimal`, `z_crosstalk.py:297-337`)
 - 跑 `TransientSensingExperiment` (`qubit=qubit_B_at_opt`, `flux_signal=phi_B_true`)
 - 用 `TransientReconstruction` (`method="wiener"`) 反演 → `phi_B_reconstructed`
3. 频域反卷积 H_BA (`z_crosstalk.py:186-225`):

$$\hat{H}_{BA}(\omega) = \frac{\hat{\Phi}_B(\omega) \cdot V_A^*(\omega)}{|V_A(\omega)|^2 + \lambda^2}$$

```
Phi_B_omega = np.fft.fft(phi_B_resampled)
V_A_omega = np.fft.fft(V_A_arr)
H_BA = Phi_B_omega * np.conj(V_A_omega) / (np.abs(V_A_omega) ** 2 + lam**2)
```

4. 真值对比 ——`H_BA_true = true_transfer_matrix.H_ji(QB, QA)`, 计算 `fit_error_dB`
5. 补偿验证 (`z_crosstalk.py:249-291`):
 - 寄生相位 $|_B_true| dt$ (未补偿)
 - 补偿后残差 $_B_true - _B_reconstructed$
 - `compensation_factor = 未补偿 / 已补偿`

返回 dict 含 `phi_A`, `phi_B_true`, `phi_B_reconstructed`, `H_BA_estimated`, `H_BA_true`, `fit_error_dB`, `parasitic_phase_uncompensated/compensated`, `compensation_factor`, `phi_B_after_compensation`.

3.6 Notebook 端到端示例 (§10)

`Simulation_sqc.ipynb` §10 是一套完整的预失真验证流程, 5 个子章节:

3.6.1 §10.0 —设置 (cell 24)

```
qubit = TransmonQubit(EC=0.2*2*np.pi, EJ=10.0*2*np.pi,
                      T1=100e3, T2=50e3, flux=0.0, n_levels=2)
```

故意夸大失真以可视化清晰 (真实系统 $amp \sim 1-10\%$)

```
dist = SingleExponentialDistortion(amplitude=0.3, tau=30.0)
```

```
line = ControlLine(name="Z0", kind="z", source="AWG0:CHO", target="Q0",
                  transfer_function=dist)
```

3.6.2 §10.1 —Rabi Chevron 失真诊断 (cell 26)

跑两个 Chevron (detuning $\times$ pulse duration 的 P_e 二维图):

- 理想: 解析公式 $p_e = (\Omega/\Omega_{\text{eff}})^2 \cdot \sin^2(\frac{1}{2}\Omega_{\text{eff}}t)$
- 失真: 用 `dist.apply(0mega-ones, dt)` 得失真后的 Rabi 包络, 再 `mesolve`

差分图 (b-a) 揭示失真在 $|\Delta| \Omega$ 附近最显著。附带证据: 累积 $\Omega(t)dt$ vs t 的曲线在初期更平、晚期赶上——这正是阶跃响应的镜像。

3.6.3 §10.2 —两种阶跃响应测量方法

- §10.2.1 Cryoscope (cell 29): 用 §1.2 的 pipeline 测 step. flux=0.25 敏感点 + step_amp=0.001 + tau=50ns.
- §10.2.2 Transient (cell 31): 用 §1.5 的 pipeline 测同一个 step. lambda_reg=50.0 (增加正则避免 ringing).
- §10.2.3 对比 (cell 33): 可视化 RMSE、上升时间、采样代价。

3.6.4 §10.3 —预失真滤波器设计 (cell 35)

三种方法并跑:

```
designer_iir = PredistortionDesigner(method="iir_inverse")
designer_fir = PredistortionDesigner(method="fir_inverse", n_taps=32, regularization=1e-3)
designer_freq = PredistortionDesigner(method="frequency_inverse", regularization=1e-4)
```

```
inv_iir = designer_iir.design(dist, dt=dt)
inv_fir = designer_fir.design(dist, dt=dt)
inv_freq = designer_freq.design(dist, dt=dt)
```

6 张图诊断: - (a) $|H(\cdot)|$ 原始 - (b) $|H_{\text{inv}}(\cdot)|$ 三方法 - (c) 级联 $|H_{\text{inv}} \cdot H|$ 1 验证 (IIR 在所有频段几乎完美 = 1, FIR 高频偏离) - (d) 级联相位 0 验证 - (e) FIR taps 图 (看尾部能量分布) - (f) 预失真后阶跃响应: 理想 / 未校正 / IIR 校正 / FIR 校正

定量: 未校正 RMSE $\rightarrow$ IIR 校正 RMSE 提升 $\sim 50\text{--}100\times$ (notebook 实测)。

3.6.5 §10.4 —三层验证 (cells 38, 40, 42)

§10.4.1 阶跃响应再次 Cryoscope 测量 (cell 38):

预失真 AWG $\rightarrow$ 失真线 $\rightarrow$ 片上 $\rightarrow$ Cryoscope 测出 h_{cryo2} , 对比未补偿的 h_{cryo} . RMSE before/after 改善因子直接量化。

§10.4.2 Chevron 复检 (cell 40):

预失真后的驱动 Chevron 图 (b) vs 未补偿 (a) vs 理想 (c): 肉眼可见 (b) 几乎与 (c) 一致。

§10.4.3 端到端 workflow (cell 42):

```
target = Waveform(t_list=t_wf,
                  samples=np.where((t_wf > 30) & (t_wf < 100), 1.0, 0.0))
wf = PredistortionValidationWorkflow(
    target_waveform=target,
    true_distortion=dist,
    designer=PredistortionDesigner(method="auto"),
)
result = wf.run()
m = result["metrics"]
print(f"RMSE: {m['rmse_uncorrected']:.6f}  $\rightarrow$  {m['rmse_corrected']:.6f}")
print(f"Settling: {m['settling_uncorrected_ns']:.1f}  $\rightarrow$  {m['settling_corrected_ns']:.1f} ns")
```

notebook 实测的 RMSE 改善因子 $\sim 30\text{--}100\times$, settling time 从 ~ 200 ns $\rightarrow$ 0 ns (在容差 $1e\text{--}3$ 内)。

3.7 约束与陷阱

约束	出处	说明
单指数 IIR 逆解析最优	§3.3.2.A	但多指数时不稳定, 必须走频域
FIR 逆需 N_{taps}/dt	物理	否则截断误差大, notebook 用 64
regularization 决定噪声/精度权衡	waveform.py:323	默认 $1e\text{--}4$ 偏激进; 噪声大时调到 $1e\text{--}3$
check_pole_stability 必须做	§3.3.3	不稳 IIR 在某些 (b, a) 下会发散
Predistortion 必须在 AWG dt 同一时基	waveform.py:281	否则采样不齐, 加重 aliasing
simulation method == transfer_function	waveform.py:90	alias 保留, 未来可能合并

约束	出处	说明
Z-crosstalk 要 QB 在 sensitivity 最大点	z_crosstalk.py:299-321	否则 Δ 太小测不出

附录

附录 A • 公式速查

量	公式	代码位置
Transmon $f(\Phi)$	$\omega_Q = \sqrt{8E_J} \cos(\pi\Phi) \overline{E_C} - E_C$	src/qubit.py <code>TransmonQubit.frequency</code>
Transmon $\Phi(\)$ 反演	$\Phi = \pi^{-1} \arccos[(f + E_C)^2 / (8E_J E_C)]$	reconstruction/dispersion.py:11
Ramsey 相位	$\varphi(\tau) = \int_0^\tau \Delta\omega(t) dt$	hardware/readout.py:97
IQ 相位提取	$\varphi = \text{atan2}(\frac{1}{2} - p_e^I, p_e^Q - \frac{1}{2})$	experiments/cryoscope.py:117
Cryoscope d/dt	$d\varphi/dt = \Delta\omega(h(t))$	reconstruction/cryoscope.py:86
Delay Ramsey 斜率	$\varphi_{\text{cal}}(z) = \tau_R \kappa z$	reconstruction/delay_ramsey.py:162
pulse comp	$\Phi_{\text{tail}}(\tau) = -z^*(\tau)$	reconstruction/pi_pulse_comp.py:43
Wiener 反卷积	$\hat{B}(\omega) = \frac{H^* \Delta P}{ H ^2 + \lambda^2}$	reconstruction/transient.py:149
Hammerstein 反演	Wiener $\rightarrow$ $\rightarrow$ 解析 Φ	reconstruction/transient.py:163
LM 更新	$\Delta b = (J^T J + \mu I + \lambda R)^{-1} J^T r$	reconstruction/transient.py:461
Adjoint Jacobian	$J_{ik} = \int \text{Tr}[\lambda(t)[G, \rho]] \partial \Delta\omega / \partial \Phi \cdot \phi_k dt$	reconstruction/transient.py:301
Secant 法	$V_{n+1} = V_n - r_n \frac{V_n - V_{n-1}}{r_n - r_{n-1}}$	calibration/frequency.py:374
Bisection 误差	$ r_n \sim (V_b - V_a) / 2^n$	calibration/frequency.py:437
Transient frequency	$\Delta\omega = p_{\text{diff}} / G_\alpha, G_\alpha = \int k(t) dt$	calibration/frequency.py:567-581
单指数失真	$H(\omega) = (1 - \alpha) + \alpha / (1 + j\omega\tau)$	hardware/distortion.py:177
单指数解析逆	$H^{-1}(s) = (1 + s\tau) / (1 + s\tau(1 - \alpha))$	calibration/waveform.py:373
频域 Wiener 逆	$H^{-1} = \frac{H^*}{ H ^2 + \lambda^2}$	calibration/waveform.py:432
Z-crosstalk H_{BA}	$H_{\text{BA}}(\omega) = \Phi_B(\omega) V_A^* / (V_A ^2 + \lambda^2)$	workflows/z_crosstalk.py:223

附录 B • 参考文献

1. **Gao et al. 2021** — *A Practical Guide for Building Superconducting Quantum Devices*. PRX Quantum 2, 040202. 文件: [idea/refactor/Gao 等 - 2021 - Practical Guide for Building Superconducting Quantum Devices.pdf](#)
 - §III.B — Transmon 参数推荐范围
 - §III.D — 控制线传递函数
 - §V.B — Ramsey 相位累积
 - §V.E — Cryoscope distortion 标定
2. **Rol et al. 2020** — *Time-domain characterization and correction of on-chip distortion of control pulses in a quantum processor*. Appl. Phys. Lett. 116, 054001. (Cryoscope 原始论文)
3. **Vepsäläinen et al. 2022** — *Improving qubit coherence using closed-loop feedback*. Nat. Commun. 13, 1932. (Closed-loop frequency calibration 参考)
4. **Kelly et al. 2018** — *Physical qubit calibration on a directed acyclic graph*. arXiv:1803.03226. (Calibration DAG 设计参考; scheduler 的 `check_state/maintain/auto_calibrate` 是这套设计的 stub)

辅助资料:

- [note/_sensing_theory.md](#) — 项目内部物理理论笔记, 含瞬态磁场协议、核函数策略推导
- [idea/refactor/_refactor_plan.md](#) — sqc/ 框架重构主方案
- [docs/architecture.md](#) — 框架架构总览

附录 C • 测试与回归索引

单元测试 (tests/unit/)

模块	测试文件
Cryoscope Experiment	test_cryoscope_experiment.py
Delay Ramsey Experiment	test_delay_ramsey_experiment.py
-pulse Comp Experiment	test_pi_pulse_comp_experiment.py
LM Reconstruction	test_lm_reconstruction.py
Basis Functions	test_basis_module.py
Kernel Estimator	test_kernel_estimator.py
IQ Readout	test_iq_readout.py
Calibration Table	test_calibration_table.py
Distortion Models	test_distortion_models.py
Predistortion Designer	test_predistortion_designer.py
Control Line	test_control_line.py
Transfer Matrix	test_transfer_matrix.py
Chip Topology	test_chip_topology.py

回归测试 (tests/regression/)

测试	内容
test_physics_baseline.py	物理回归 baseline (rtol=1e-6, atol=1e-9, 不可放宽)
test_predistortion_baseline.py	预失真端到端 baseline
test_z_crosstalk_baseline.py	Z-crosstalk workflow baseline
generate_baselines.py	baseline 生成入口 (修改 CONFIG 后必须重跑)

运行测试

完整测试套件

```
"C:\Users\21034\anaconda3\envs\qutip-env\python.exe" -m pytest tests/ -v
```

仅回归

```
"C:\Users\21034\anaconda3\envs\qutip-env\python.exe" -m pytest tests/regression -m regression
```

单个模块

```
"C:\Users\21034\anaconda3\envs\qutip-env\python.exe" -m pytest tests/unit/test_cryoscope_experiment.py -v
```

附录 D • CONFIG 默认值 (关键超参)

字段	默认值	影响
CONFIG.awg.sample_rate	2.0 GSa/s	dt = 0.5 ns 派生
CONFIG.pulse.t_rabi_duration	10.0 ns	脉冲长度
CONFIG.pulse.t_global_end	400.0 ns	仿真窗口
CONFIG.transmon.EC	0.2 GHz·(2)	$E_C/h = 200$ MHz
CONFIG.transmon.EJ	15.0 GHz·(2)	$E_J/h = 15$ GHz
CONFIG.reconstruction.lambda_reg	10.0	Wiener 正则化
CONFIG.reconstruction.stim_amplitude	0.0215	kernel 探针
CONFIG.reconstruction.stim_width	3.0 ns	kernel 探针宽度
CONFIG.reconstruction.lm_n_basis	100	LM 基函数数
CONFIG.reconstruction.lm_basis_type	"fourier"	LM 基类型

字段	默认值	影响
CONFIG.reconstruction.lm_lambda	100.0	LM 平滑正则化
CONFIG.reconstruction.cryoscope_tau	100.0 ns	Cryoscope 标定
CONFIG.reconstruction.delay_ramsey_tau	20.0 ns	Delay Ramsey __R
CONFIG.reconstruction.pi_pulse_T_pi	10.0 ns	pulse 宽度

修改 CONFIG 后: 必须重新生成 baseline (tests/regression/generate_baselines.py), 并在 commit message 明确标注 (CLAUDE.md R9)。

文档版本 *v1.0* 生成日期 *2026-05-14* 维护: *sqc/* 框架重构 *Track A*